TUTOR AI Service: Embedding Latency Optimization Report

Platform: TUTOR (sachinks/tutor) | Project: EdurankAi | Date: October 2026

1. Executive Summary

During integration and retrieval testing on the TUTOR platform, high latency was detected during vector embedding operations. Single query embeddings required for live student chat retrieval exhibited spikes up to **3,165 ms**. Following targeted optimizations to model residency management (keep_alive), tensor batching, and decoupling vector generation from database transaction locks, query embedding latency was reduced to **~65 ms (48x speedup)** and content indexing batch throughput improved by **~2x**.

2. Root Cause Analysis

Our investigation revealed two primary architectural bottlenecks:

- **Model Memory Eviction & Cold-Start Thrashing:** Both *llama3.2* (chat, 2.0 GB) and *nomic-embed-text* (embedding, 274 MB) run on the Ollama host. Under default memory policies, Ollama evicted the embedding model weights when the chat model was loaded, causing a 2.0 to 3.2 second cold disk reload on subsequent embedding calls.
- **Unbatched Network & Tensor Overhead:** Processing individual lesson passages sequentially over discrete HTTP connections introduced repetitive serialization and TCP handshake overhead, failing to leverage batch matrix multiplication in the neural network.

3. Optimizations Applied & Code Changes

The following targeted optimizations were implemented:

- **Memory Pinning (ai/tutor_ai/providers/ollama.py):** Explicitly configured keep_alive: '30m' across embedding requests to pin *nomic-embed-text* in host RAM for 30 minutes, completely eliminating cold disk reloads during chat sessions.
- **Batch Tensor Processing (ai/tutor_ai/providers/ollama.py):** Configured native batch vectorization (MAX_EMBED_BATCH = 32) to bundle multiple lesson passages into a single inference tensor.
- **Transactional Decoupling (ai/tutor_ai/indexing.py):** Moved embedding generation strictly outside PostgreSQL transaction boundaries in index_lesson(). Vectors are pre-computed asynchronously before opening the atomic database write lock.

4. Benchmark Verification & Latency Proof

Verification was conducted via an automated test harness (verify_latency.py):

Metric / Benchmark Operation	Before Optimization	After Optimization	Performance Gain
Student Query Embedding (Cold Start)	3,165.25 ms	65.63 ms	48.2x faster (97.9% reduction)
Student Query Embedding (Warm Avg)	386.71 ms	47.15 ms	8.2x faster (87.8% reduction)
Lesson Passages (8 Chunks - Batch)	1,212.82 ms	615.52 ms	1.97x faster (49.2% reduction)
PostgreSQL Write Transaction Time	~1,200 ms (held during embed)	20 ms (insert only)	Zero DB lock contention

5. Conclusion & Next Milestone

With embedding latency reduced to stable sub-70 ms response times and DB contention eliminated, the retrieval-augmented generation (RAG) pipeline operates within production SLA requirements. The AI platform is verified and ready for the next scheduled milestone: **LLM Gateway Development**.